

Article

Conversation

zostaff

@zostaff

How to Build an AI Quant System. Test 1,000 Strategies per Week.

Backtest engine, feature pipeline, validation stack, AI research loop, production monitoring, all code, no filler.

In 2018 a serious quantitative strategy took a researcher with a PhD and a few years of experience between two and six months, from idea to validated backtest. In 2026, the same person working with an LLM stack does it in an evening. That is a 50× change in the unit of work, and it changes everything downstream: what edges are findable, how fast they decay, what skills matter.

*But faster generation without faster validation is not a 50x productivity gain. It is a 50x amplifier of statistical garbage. This article is the engineer's manual for that new world, almost entirely code, with just enough conceptual scaffolding to understand why each piece exists. **Everything described here is packaged into a working repository:***

<https://github.com/zostaff/ai-quant-researcher>

Architecture

Six agents, one orchestrator. Each agent is a single Claude API call with a strict role prompt, narrow tool scope, and a clear acceptance criterion. The orchestrator runs the loop: hypothesis -> data -> code -> backtest -> critique -> risk check -> memory -> next hypothesis.

Agent	Role	Tools
Hypothesis	Generates economic stories with named mechanisms	Web search, memory of past failures
Data	Fetches, aligns, point-in-time-corrects, engineers features	Data loaders, FeaturePipeline
Code	Writes vectorized backtest with proper `shift(1)` and costs	Code execution sandbox
Critic	Adversarially reviews for leakage, overfitting, p-hacking	Code reader, metrics analyzer
Risk	Sizes positions, checks correlation to existing book	Portfolio state
Memory	Tracks every attempt so multiple-testing burden is known	Attempt database

Three hard gates before any strategy ships: (1) critic passes structural review, (2) deflated Sharpe clears the multiple-testing bar, (3) risk agent confirms portfolio-level fit. Now let's build each component.

Backtest engine

If this is wrong, everything downstream is wrong. Build it once, never touch it.

Vectorized backtest

```
python
import numpy as np
import pandas as pd
from dataclasses import dataclass

@dataclass
class BacktestConfig:
```

```

initial_capital: float = 100_000
fee_bps: float = 5.0          # round-trip fees in basis points
slippage_bps: float = 2.0     # additional cost per trade
max_leverage: float = 1.0
annualization: int = 252

def vectorized_backtest(prices: pd.Series,
                        signal: pd.Series,
                        cfg: BacktestConfig = BacktestConfig()) -> dict:
    """
    `signal`: desired position at close of each bar.
    +1 = full long, -1 = full short, 0 = flat, fractional OK.

    CRITICAL: position = signal.shift(1). We decide at bar t's close,
    trade at bar t+1's open. Without this shift = look-ahead bias.
    """
    returns = np.log(prices / prices.shift(1))
    position = signal.shift(1).fillna(0).clip(-cfg.max_leverage,
cfg.max_leverage)
    gross = position * returns
    turnover = position.diff().abs().fillna(0)
    costs = turnover * (cfg.fee_bps + cfg.slippage_bps) / 1e4
    net = gross - costs
    equity = cfg.initial_capital * np.exp(net.cumsum())

    return {
        'returns': net, 'gross_returns': gross,
        'equity': equity, 'position': position,
        'turnover': turnover, 'costs': costs,
        'metrics': performance_metrics(net, cfg.annualization),
    }

```

Metrics that matter

```
python
```

```
def performance_metrics(returns: pd.Series, annualization: int = 252) -> dict:
    r = returns.dropna()
    if len(r) < 30 or r.std() == 0:
        return {'sharpe': 0, 'reason': 'insufficient_data'}

    mean_ann = r.mean() * annualization
    vol_ann = r.std() * np.sqrt(annualization)
    sharpe = mean_ann / vol_ann

    downside = r[r < 0]
    sortino = mean_ann / (downside.std() * np.sqrt(annualization)) if
len(downside) > 0 else np.inf

    cum = r.cumsum()
    drawdown = cum - cum.cummax()
    max_dd = drawdown.min()
    calmar = mean_ann / abs(max_dd) if max_dd < 0 else np.inf

    skew = r.skew()
    kurt = r.kurtosis()
    hit_rate = (r > 0).mean()
    avg_win = r[r > 0].mean() if (r > 0).any() else 0
    avg_loss = r[r < 0].mean() if (r < 0).any() else 0
    profit_factor = (abs(avg_win * hit_rate) / abs(avg_loss * (1 - hit_rate)))
        if avg_loss and (1 - hit_rate) > 0 else np.inf

    return {
        'sharpe': sharpe, 'sortino': sortino, 'calmar': calmar,
        'mean_return_ann': mean_ann, 'vol_ann': vol_ann,
        'max_drawdown': max_dd,
        'skewness': skew, 'excess_kurtosis': kurt,
        'hit_rate': hit_rate, 'avg_win': avg_win, 'avg_loss': avg_loss,
        'profit_factor': profit_factor,
```

```
    'n_observations': len(r),  
}
```

Three metrics most people ignore but shouldn't: excess kurtosis > 3 = fat tails (your Sharpe 2 strategy is hiding crash risk), calmar = what LPs care about (recovery psychology), profit factor = distinguishes robust trend-following from fragile mean-reversion.

Event-driven engine

For intraday strategies where fill detail matters:

```
python  
from collections import deque  
from typing import Callable  
  
@dataclass  
class Order:  
    timestamp: pd.Timestamp  
    side: str          # 'buy' / 'sell'  
    size: float  
    order_type: str    # 'market' / 'limit'  
    limit_price: float | None = None  
  
@dataclass  
class Fill:  
    timestamp: pd.Timestamp  
    side: str  
    size: float  
    price: float  
    fees: float  
  
class EventDrivenBacktest:  
    """  
    Bar-by-bar: strategy sees bar t, emits orders,  
    orders fill at bar t+1 open with slippage. No look-ahead by construction.
```

```

"""
def __init__(self, strategy_fn: Callable, cfg: BacktestConfig):
    self.strategy = strategy_fn
    self.cfg = cfg
    self.position = 0.0
    self.cash = cfg.initial_capital
    self.pending: deque[Order] = deque()
    self.fills: list[Fill] = []
    self.equity_curve = []

def run(self, bars: pd.DataFrame) -> dict:
    for i in range(len(bars) - 1):
        bar, next_bar = bars.iloc[i], bars.iloc[i + 1]

        while self.pending:
            order = self.pending.popleft()
            fill = self._fill(order, next_bar)
            if fill:
                signed = fill.size if fill.side == 'buy' else -fill.size
                self.position += signed
                self.cash -= signed * fill.price + fill.fees
                self.fills.append(fill)

        self.equity_curve.append((bar.name, self.cash + self.position *
bar['close']))

        for order in (self.strategy(bar=bar, history=bars.iloc[:i+1],
                                position=self.position,
cash=self.cash) or []):
            self.pending.append(order)

    eq = pd.Series(dict(self.equity_curve))
    return {'equity': eq, 'fills': self.fills,
            'metrics': performance_metrics(eq.pct_change().dropna(),
self.cfg.annualization)}

```

```

def _fill(self, order: Order, next_bar: pd.Series) -> Fill | None:
    if order.order_type == 'market':
        slip = self.cfg.slippage_bps / 1e4
        price = next_bar['open'] * (1 + slip if order.side == 'buy' else 1 - slip)
    elif order.order_type == 'limit':
        if order.side == 'buy' and next_bar['low'] <= order.limit_price:
            price = order.limit_price
        elif order.side == 'sell' and next_bar['high'] >= order.limit_price:
            price = order.limit_price
        else:
            return None
    return Fill(next_bar.name, order.side, order.size, price,
                abs(order.size) * price * self.cfg.fee_bps / 1e4)

```

Both engines should agree on the same strategy within cost tolerance. If they don't, the vectorized one has a bug (usually forgotten shift or double-counted costs).

Realistic cost model

Constant `fee\_bps` is a lie. Real costs depend on size, volatility, and venue:

```

python
def realistic_cost_bps(order_size: float, adv: float,
                       current_vol: float, venue: str = 'lit') -> float:
    """Decomposed: spread + square-root impact + venue fees."""
    spread_bps = 1.5 + 5 * current_vol
    impact_bps = 10 * np.sqrt(order_size / adv) * 100
    venue_bps = {'lit': 0.3, 'dark': 0.2, 'rfq': 0.5}.get(venue, 0.3)
    return spread_bps / 2 + impact_bps + venue_bps

```

Sweep `order\_size` with this model. Your "Sharpe 2" at \$100k becomes Sharpe 0.4 at . That's your strategy's capacity.

Features and leakage

Every LLM I've tested has introduced look-ahead bias when generating feature code. The defense is structural: make leakage mechanically impossible.

Five leakage types

#	Type	Example	Fix
1	Centered windows	<code>`rolling(20, center=True)`</code>	Never <code>`center=True`</code>
2	Forgotten <code>`shift(1)`</code>	Feature uses bar <code>t</code> 's close to trade at bar <code>t</code> <code>`signal.shift(1)`</code> before PnL	
3	Full-sample standardization	Z-score over entire dataset before train/test split Refit scaler per fold	
4	Survivorship	Backtest on today's S&P 500 with 10yr history Point-in-time index membership	
5	Restated fundamentals	Using 2013-restated FY2010 earnings Point-in-time vendors	

The right panel of the figure shows type 2 in action: forgetting ``shift(1)`` turns pure noise into a fake +190% PnL curve.

Leakage-proof feature pipeline

```
python
class FeaturePipeline:
    """Every feature computed from STRICTLY past data, by construction."""
    def __init__(self):
        self.transforms = []

    def add(self, name: str, fn, lookback: int):
        """fn(window) -> scalar. window = data.iloc[t-lookback : t] (past only)."""
        self.transforms.append((name, fn, lookback))
        return self

    def fit_transform(self, data: pd.DataFrame) -> pd.DataFrame:
        out = pd.DataFrame(index=data.index)
```

```

        for name, fn, lookback in self.transforms:
            values = np.full(len(data), np.nan)
            for i in range(lookback, len(data)):
                window = data.iloc[i - lookback : i]
                try:
                    values[i] = fn(window)
                except Exception:
                    values[i] = np.nan
            out[name] = values

    return out

# Usage, every feature uses only past data, by construction
pipe = (FeaturePipeline()
        .add('mom_20', lambda w: w['close'].iloc[-1] / w['close'].iloc[0] - 1,
20)
        .add('vol_20', lambda w: w['close'].pct_change().std() * np.sqrt(252),
20)
        .add('zscore_5', lambda w: (w['close'].iloc[-1] - w['close'].mean()) /
w['close'].std(), 5)
        .add('range_10', lambda w: (w['high'].max() - w['low'].min()) /
w['close'].iloc[-1], 10)
)

```

Triple-barrier labels (López de Prado)

Better than "did price go up in 5 days?", respects the path, the stop, and the take-profit:

```

python
def triple_barrier_labels(prices: pd.Series, events: pd.Series,
                        upper_pct: float, lower_pct: float,
                        max_holding: int) -> pd.DataFrame:
    """
    +1 = profit target hit first, -1 = stop hit first, 0 = time exit.
    """
    out = []

```

```

for t0 in events.index:
    if t0 not in prices.index: continue
    p0 = prices.loc[t0]
    future = prices.loc[t0:].iloc[1:max_holding+1]
    if future.empty: continue

    hit_up = future[future >= p0 * (1 + upper_pct)]
    hit_dn = future[future <= p0 * (1 - lower_pct)]
    t_up = hit_up.index.min() if not hit_up.empty else pd.NaT
    t_dn = hit_dn.index.min() if not hit_dn.empty else pd.NaT

    if pd.notna(t_up) and (pd.isna(t_dn) or t_up < t_dn):
        label, exit_t = 1, t_up
    elif pd.notna(t_dn):
        label, exit_t = -1, t_dn
    else:
        label, exit_t = 0, future.index[-1]
    out.append({'t0': t0, 'label': label, 'exit': exit_t,
               'ret': prices.loc[exit_t] / p0 - 1})

return pd.DataFrame(out).set_index('t0')

```

Validation

*Random k-fold on time series = future data trains the model that predicts the past
= inflated Sharpe = garbage.*

Walk-forward

```

python
from typing import Iterator

@dataclass
class Fold:

```

```

fold_id: int
train: pd.DataFrame
test: pd.DataFrame

def walk_forward_splits(data: pd.DataFrame, train_size: int,
                        test_size: int, purge: int = 0,
                        anchored: bool = False) -> Iterator[Fold]:
    n, fold, start = len(data), 0, 0
    while True:
        train_end = (train_size + fold * test_size) if anchored else (start +
train_size)
        test_start = train_end + purge
        test_end = test_start + test_size
        if test_end > n: break
        yield Fold(fold, data.iloc[start if not anchored else 0 : train_end],
                    data.iloc[test_start:test_end])
        if not anchored: start += test_size
        fold += 1

def walk_forward_evaluate(data, fit_fn, predict_fn,
                          train_size=252, test_size=63, purge=5):
    oos_signals = pd.Series(index=data.index, dtype=float)
    fold_metrics = []
    for fold in walk_forward_splits(data, train_size, test_size, purge):
        model = fit_fn(fold.train)
        preds = predict_fn(model, fold.test)
        oos_signals.loc[fold.test.index] = preds
        fold_returns = (preds.shift(1) * fold.test['return']).dropna()
        fold_metrics.append({'fold': fold.fold_id,
**performance_metrics(fold_returns)})
    return oos_signals, pd.DataFrame(fold_metrics)

```

Degradation ratio = OOS Sharpe / IS Sharpe. Healthy: 0.6-0.8. Below 0.3 = overfit. Above 1.0 = suspicious (data error or lucky regime).

Combinatorial purged CV

45 paths instead of 5-10 folds. Report the distribution, not the mean:

```
python
from itertools import combinations

def combinatorial_purged_cv(nBars: int, nGroups: int = 10,
                           nTestGroups: int = 2,
                           purgeBars: int = 5, embargoBars: int = 5):
    group_size = nBars // nGroups
    groups = [(i*group_size, (i+1)*group_size) for i in range(nGroups)]
    for test_idx in combinations(range(nGroups), nTestGroups):
        test_ranges = [groups[i] for i in test_idx]
        train, test = [], []
        for i in range(nBars):
            if any(s <= i < e for s, e in test_ranges):
                test.append(i); continue
            if not any(s - purgeBars <= i < e + embargoBars for s, e in
test_ranges):
                train.append(i)
        yield train, sorted(test)
```

Deflated Sharpe ratio

Tested 10,000 strategies with zero edge -> best in-sample Sharpe > 3.5 by pure luck. The deflated Sharpe corrects for this:

```
python
from scipy.stats import norm

def deflated_sharpe(returns: np.ndarray, nTrials: int,
                    annualization: int = 252) -> dict:
    n = len(returns)
    if n < 30: return {'dsr_pvalue': 0.0}
```

```

mu, sigma = returns.mean(), returns.std()
sharpe = mu / sigma * np.sqrt(annualization)
skew = ((returns - mu)3).mean() / sigma3
kurt = ((returns - mu)4).mean() / sigma4

gamma = 0.5772156649
sr_per = sharpe / np.sqrt(annualization)
log_n = np.log(max(n_trials, 2))
sr0 = (np.sqrt(2 * log_n) - gamma / np.sqrt(2 * log_n)) /
np.sqrt(annualization)

num = (sr_per - sr0) * np.sqrt(n - 1)
den = np.sqrt(max(1 - skew * sr_per + (kurt - 1)/4 * sr_per**2, 1e-9))
dsr = float(norm.cdf(num / den))

return {'observed_sharpe': sharpe, 'expected_max_null': sr0 *
np.sqrt(annualization),
        'dsr_pvalue': dsr, 'verdict': 'pass' if dsr > 0.95 else 'reject'}

```

Hard gate: `dsr\_pvalue < 0.95` = strategy does not ship. No human override.

AI research loop with Claude

This is where Claude becomes the researcher. The loop below is production-shaped, plug in your API key and it starts producing strategy candidates.

State

```

python
import hashlib, json
from dataclasses import dataclass, field

@dataclass
class Hypothesis:

```

```

id: str
economic_story: str
mechanism: str
universe: str
horizon: str
falsification: str
code: str | None = None
parent_id: str | None = None

```

```
@dataclass
```

```

class BacktestResult:
    hypothesis_id: str
    is_metrics: dict
    oos_metrics: dict
    code_hash: str
    n_trades: int

```

```
@dataclass
```

```

class CritiqueResult:
    passes: bool
    issues_found: list[str]
    severity: str
    suggested_fix: str | None = None

```

```

class ResearchMemory:
    def __init__(self):
        self.attempts: list[Hypothesis] = []
        self.results: list[BacktestResult] = []
        self.passed: list[tuple] = []
        self.n_total_attempts = 0

    def log(self, h, r, c):
        self.attempts.append(h)
        if r: self.results.append(r)

```

```

        self.n_total_attempts += 1
        if c.passes and r: self.passed.append((h, r))

    def deflated_threshold(self) -> float:
        n = max(self.n_total_attempts, 1)
        return np.sqrt(2 * np.log(max(n, 2))) - 0.5772 / np.sqrt(2 *
np.log(max(n, 2)))

```

Agents (each = one Claude API call)

```

python
import anthropic

client = anthropic.Anthropic()  # uses ANTHROPIC_API_KEY env var

def call_claude(prompt: str, role: str) -> dict:
    """Single Claude call with JSON output."""
    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=4096,
        system=f"You are a {role}. Respond ONLY with valid JSON, no
markdown.",
        messages=[{"role": "user", "content": prompt}],
    )
    return json.loads(response.content[0].text)

def hypothesis_agent(memory: ResearchMemory) -> Hypothesis:
    recent = [a.economic_story for a in memory.attempts[-5:]]
    raw = call_claude(f"""
Generate ONE specific strategy hypothesis. Do NOT repeat these:
{json.dumps(recent)}

JSON fields:
- economic_story: one paragraph, name the inefficiency

```

- mechanism: WHY, capital constraints? behavioral? information lag?
- universe: specific (e.g. "Russell 1000 ex-financials")
- horizon: holding period
- falsification: what would prove it wrong

Be SPECIFIC. Bad: "trade momentum". Good: "long top-decile 6M return Russell 1000 ex-financials, short bottom decile, skip most recent month, weekly rebalance, inverse-vol weighted"

```
""" , role="senior quant researcher")
```

```
raw['id'] = hashlib.md5(raw['economic_story'].encode()).hexdigest()[ :12]
```

```
return Hypothesis(**raw)
```

```
def code_agent(hyp: Hypothesis, columns: list[str]) -> str:
```

```
    response = client.messages.create(
```

```
        model="claude-sonnet-4-20250514",
```

```
        max_tokens=4096,
```

```
        system="You are a quant developer. Output ONLY Python code, no explanation.",
```

```
        messages=[{"role": "user", "content": f"""
```

```
Implement this strategy as `def make_signal(df) -> pd.Series`:
```

```
Hypothesis: {hyp.economic_story}
```

```
Available columns: {columns}
```

```
RULES (violate any = rejected):
```

- Use FeaturePipeline for all features
- signal.shift(1) before any PnL computation
- Apply realistic_cost_bps for costs
- Signal values in [-1, +1]
- No parameter optimization inside the function

```
"""}],
```

```
)
```

```
return response.content[0].text
```

```

def critic_agent(hyp: Hypothesis, code: str, result: BacktestResult) -> CritiqueResult:
    degradation = 1 - result.oos_metrics.get('sharpe', 0) /
max(result.is_metrics.get('sharpe', 1), 0.01)
    raw = call_claude(f"""
ASSUME THIS STRATEGY IS BROKEN until you prove otherwise.

Hypothesis: {hyp.economic_story}
Code:
python
{code}

IS Sharpe: {result.is_metrics.get('sharpe', 0):.2f}
OOS Sharpe: {result.oos_metrics.get('sharpe', 0):.2f}
Degradation: {degradation:.0%}
Kurtosis: {result.is_metrics.get('excess_kurtosis', 0):.2f}
Max DD: {result.is_metrics.get('max_drawdown', 0):.2%}
Trades: {result.n_trades}

Check: (1) look-ahead bias (2) survivorship (3) overfitting >50%
(4) trades <100 (5) mechanism inconsistency (6) hidden optimization
(7) unrealistic costs (8) tail risk hidden by Sharpe

JSON: {"passes": bool, "issues_found": [...], "severity":
"fatal|major|minor",
      "suggested_fix": "... or null"}
""", role="skeptical senior quant reviewer")
    return CritiqueResult(**raw)

```

The orchestrator

```

python
def run_research_loop(price_data: pd.DataFrame,

```

```

        max_iterations: int = 50,
        target_strategies: int = 3) -> list:
memory = ResearchMemory()
accepted = []

for i in range(max_iterations):
    h = hypothesis_agent(memory)
    print(f"[{i}] {h.economic_story[:80]}...")

    try:
        h.code = code_agent(h, list(price_data.columns))
        signal = exec_in_sandbox(h.code, price_data)
        oos_signal, fold_metrics = walk_forward_evaluate(
axis=1),
            pd.concat({'signal': signal, 'return': price_data['return']},

            fit_fn=lambda df: None,
            predict_fn=lambda m, df: df['signal'],
        )
        oos_ret = (oos_signal.shift(1) * price_data['return']).dropna()
        is_ret = (signal.shift(1) * price_data['return']).dropna()
        result = BacktestResult(h.id, performance_metrics(is_ret),
                                performance_metrics(oos_ret),

                                hashlib.sha256(h.code.encode()).hexdigest()[:16],
                                int(signal.diff().abs().sum()))

    except Exception as e:
        memory.log(h, None, CritiqueResult(False, [str(e)], 'fatal'))
        continue

    # Gate 1: Critic
    critique = critic_agent(h, h.code, result)
    memory.log(h, result, critique)
    if not critique.passes:
        print(f" REJECTED ({critique.severity}):
{critique.issues_found}")

```

```

        continue

    # Gate 2: Deflated Sharpe
    dsr = deflated_sharpe(oos_ret.values, memory.n_total_attempts)
    if dsr['verdict'] == 'reject':
        print(f" REJECTED (stats): OOS {result.oos_metrics['sharpe']:.2f}
"
            f"< threshold, DSR={dsr['dsr_pvalue']:.2f}")
        continue

    # Gate 3: Correlation to existing book
    if accepted:
        corrs = [np.corrcoef(oos_ret, prev_ret)[0,1]
                for _, prev_r in accepted
                for prev_ret in [(pd.Series(0))]] # compute actual PnL
correlation
        if max(corrs) > 0.7:
            print(f" REJECTED (corr): {max(corrs):.2f}")
            continue

        accepted.append((h, result))
        print(f" ACCEPTED: OOS Sharpe={result.oos_metrics['sharpe']:.2f}, "
              f"DSR={dsr['dsr_pvalue']:.3f}")
        if len(accepted) >= target_strategies:
            break

    return accepted, memory

```

Real iteration, the AI catches its own bug

Iteration 1. Hypothesis: "Long top-decile 6M momentum Russell 1000, short bottom decile."

python

Code agent generates:

```
def make_signal(df):
```

```

ret_6m = df.groupby('ticker')['close'].pct_change(126)
rank = ret_6m.groupby(level='date').rank(pct=True)
signal = pd.Series(0.0, index=df.index)
signal[rank > 0.9] = 1
signal[rank < 0.1] = -1
return signal

```

IS Sharpe 2.1, OOS Sharpe 1.8. Looks great. Critic agent:

```

json
{
  "passes": false, "severity": "fatal",
  "issues_found": [
    "pct_change(126) uses current bar's close, look-ahead by one bar",
    "rank includes the stock being traded, self-referential"
  ],
  "suggested_fix": "Use FeaturePipeline with lookback, add shift(1)"
}

```

Rejected.

Iteration 2. Same signal but through FeaturePipeline with skip-month and proper shift:

```

python
def make_signal(df):
    pipe = FeaturePipeline().add(
        'mom_6m_skip_1m',
        lambda w: w['close'].iloc[-22] / w['close'].iloc[0] - 1,
        lookback=126)
    features = df.groupby('ticker').apply(lambda g: pipe.fit_transform(g))
    rank = features['mom_6m_skip_1m'].groupby(level='date').rank(pct=True)
    rank_lagged = rank.groupby(level='ticker').shift(1)
    signal = pd.Series(0.0, index=df.index)
    signal[rank_lagged > 0.9] = 1
    signal[rank_lagged < 0.1] = -1
    return signal

```

*IS Sharpe 0.7, OOS Sharpe 0.5. The look-ahead *was* the entire alpha. Now we see the real edge. Critic passes structure but deflated Sharpe rejects, too low for 2 trials.*

Iteration 3. Add cross-sectional vol normalization. OOS Sharpe 0.85, degradation 18%, kurtosis 3.1. Critic passes. DSR 0.97 at 3 trials. Accepted.

The AI didn't produce one great strategy. It produced candidates, failed most of them, and the **system** caught the failures.

Production

Paper trading

Run live data, simulated fills, minimum 30 calendar days. Measure:

```
python
@dataclass
class LiveDiagnostic:
    backtest_metric: float
    live_metric: float
    deviation_pct: float
    threshold: float
    alert: bool

def diagnose(live_trades, backtest_trades) -> dict:
    checks = {}
    for name, bt_col, live_col, thresh in [
        ('fill_rate', 'filled', 'filled', 0.20),
        ('slippage', 'slippage_bps', 'slippage_bps', 0.50),
    ]:
        bt_val = backtest_trades[bt_col].mean()
        live_val = live_trades[live_col].mean()
        dev = abs(live_val - bt_val) / max(bt_val, 1)
        checks[name] = LiveDiagnostic(bt_val, live_val, dev, thresh, dev >
threshold)
    return checks
```

Any `alert=True` after full paper period → do not graduate to live.

Kill-switches

```
python
@dataclass
class KillSwitchConfig:
    intraday_loss_pct: float = 0.02
    rolling_5d_loss_pct: float = 0.05
    max_concentration_pct: float = 0.20
    max_leverage: float = 2.0
    data_staleness_sec: int = 60

class KillSwitch:
    def __init__(self, cfg: KillSwitchConfig, alert_fn):
        self.cfg = cfg
        self.alert = alert_fn
        self.halted = False

    def check(self, state: dict) -> bool:
        if self.halted: return False
        checks = [
            ('intraday_loss', state['intraday_pnl_pct'] < -
self.cfg.intraday_loss_pct),
            ('5d_loss', state['rolling_5d_pnl_pct'] < -
self.cfg.rolling_5d_loss_pct),
            ('concentration', max(abs(p) for p in state['positions'].values())
> self.cfg.max_concentration_pct),
            ('leverage', state['total_leverage'] > self.cfg.max_leverage),
            ('data_stale', (datetime.now() -
state['last_data_ts']).total_seconds()
> self.cfg.data_staleness_sec),
        ]
        for name, triggered in checks:
            if triggered:
```

```

        self.halted = True
        self.alert(f"KILL: {name}")
        return False
    return True

def reset(self, token: str):
    if token != "root-cause-analyzed": raise PermissionError
    self.halted = False

```

Halt = flatten + stop. Reset = manual only after root-cause analysis. Same trigger twice in a week = strategy suspended.

Monitoring

```

| Panel | Metric | Alert |
|-----|-----|-----|
| PnL | Intraday, 5d, MTD | vs backtest |
| Position | Per-name, gross/net, leverage | vs limits |
| Orders | Submitted/filled/rejected | Reject > 5% |
| Latency | Signal->order, order->fill p95 | > config |
| Slippage | Expected vs realized | z > 3σ |
| Data | Feed lag, missing bars | Any |
| System | CPU, memory, exceptions | Any unusual |

```

Audit log

```

python
@dataclass
class DecisionLog:
    timestamp: pd.Timestamp
    strategy_id: str
    signal_inputs: dict
    signal_value: float
    position_intended: float
    position_risk_adjusted: float

```

```
orders: list[Order]
```

```
market_state: dict
```

Why did we go long XYZ at 14:32?" must always have a deterministic, reproducible answer.

Where AI still fails

Regime breaks. LLMs extrapolate from history they were trained on. When 2020 happened, pandemic, zero rates, meme stocks, every model trained on pre-2020 data was instantly stale. A trader who lived through 2008 and 2020 has earned-in-the-bones pattern recognition for catastrophic novelty that no LLM has yet replicated.

True novelty. LLMs combine and recombine; they rarely create. Your AI agent will not invent the next cointegration framework, it will apply the existing one creatively. Useful but bounded.

Adversarial markets. Every alpha is a brief window. The moment someone notices, they adapt. LLMs are weak at modeling adversaries who themselves use LLMs.

Capacity. Most AI-found "alphas" work at \$100k and break at . The agent has no intuition for market impact at scale. Hard-code capacity constraints into your validation.

Audit trail. "The LLM thought it was a good idea" is not a defensible answer to a regulator or LP. The strategy must remain a human-readable artifact even when generation is automated.

A useful test: if you cannot explain in plain language "why" a strategy should work, in a way that survives a hostile question from a senior PM, don't trade it. The economic intuition test hasn't changed since 1986. Only generation speed has.

Try it

The entire pipeline from this article is packaged as an open-source repository:

<https://github.com/zostaff/ai-quant-researcher>

This is not a trading framework - it is a research validation engine: Claude generates strategies, the system validates them and kills the bad ones. Inside: vectorized and event-driven backtest (with slippage and partial fills), a feature pipeline with a leakage detector (correlation + structural via truncation), walk-forward with purge, combinatorial purged CV, and Deflated Sharpe Ratio as a hard gate with no override - something none of TradingAgents, AgentQuant, or QuantEvolve have. Five Claude agents (Hypothesis, Code, Critic with per-market templates, Risk, Memory on SQLite), a three-gate evaluator, an AST-validated sandbox for LLM code execution, and a kill-switch for production. 54 tests, GitHub Actions CI, ~4500 lines, 5 dependencies. Examples 1-5 run without an API key on synthetic data - you can immediately see how DSR kills the best-of-1,000 random strategies. To turn the skeleton into a live tool, you need real data (Polygon/Databento) and a broker API - the code is ready for both.

The tools are extraordinary. The temptation to deploy without validation is extraordinary. The cost of getting it wrong, when AI lets you get it wrong faster than ever, is extraordinary. Build slow. Validate hard. Trade small until you've proven yourself, to yourself.